

TEAM PLAN

ESG, Financial Performance & GAF Dataset for Stock Risk Classification

GitHub: <https://github.com/ma24ahv/Week-11-Team-3>

1. Team Roles

Each member holds a primary role (ownership) and a secondary role (support). Roles were reviewed at each sprint retrospective.

Member	Name	Primary Role	Secondary Role	GitHub
1	Tamanna	Team Leader – project coordination, sprint planning, deadline management	Model Evaluation metrics analysis and reporting across all models	tamannapathak
2	Arafat	GitHub Manager – branch strategy, PR reviews, version control	Feature Engineering – ESG and financial tabular feature pipelines	ma24ahv
3	Eben	Communication Leader – meeting minutes, stakeholder updates, liaison	GAF Image Generation timeseries to Gramian Angular Field pipeline	Princebenj
4	Josh	Scrum Master – sprint ceremonies, backlog grooming, blocker resolution	Hyperparameter Tuning – Optuna trials for CNN and ResNet50	Os25aay
5	Mim	Code Review Manager – coding standards, PR approvals, static analysis	UAT Coordination – user test script design and feedback collation	Minhajul3557
6	Sazzad	Test Plan Manager – data quality tests, unit tests, KPI validation	Literature Review – benchmarking results against published work	Sazzad2417

Team Leader (Tamanna): Oversaw the overall direction of the project, led weekly sprint planning and review meetings, coordinated team activities, monitored progress against deadlines, and was the main point of contact for resolving any issues.

GitHub Manager (Arafat): Maintained branch naming conventions, enforced PR policy (min. 1 reviewer), and tagged weekly releases on main.

Communication Leader (Eben): Organized team communication, recorded and distributed meeting minutes within 24 hours after each session, kept all members informed about project updates, and managed communication with the module tutor.

Scrum Master (Josh): Managed the project workflow using two-week sprint cycles through GitHub Projects Kanban, led sprint reviews and retrospective discussions, tracked team progress, and made sure blockers were resolved within 48 hours.

Code Review Manager (Mim): Enforced coding standards, approved PRs and ran weekly Flake8/Black linting reports.

Test Plan Manager (Sazzad): Owned the master test registry and produced the weekly KPI dashboard.

2. Team Meetings and Group Sessions

Meeting	Day/Time	Dur.	Platform	Chair
Sprint Planning & Stand-up	Monday 18:00	45 min	MS Teams (video)	Scrum Master (Josh)
Sprint Review & Retrospective	Friday 18:00	60 min	MS Teams (video)	Team Leader (Tamanna)
Ad-hoc Technical Session	As needed	30–60 min	Discord Voice	Relevant member
Assessment Milestone Review	Weeks 3, 5, 7, 9	90 min	MS Teams (video)	Team Leader (Tamanna)

Sprint Planning (Monday): Velocity review, sprint backlog selection from the GitHub Projects board, task assignment, and member commitment statements.

Sprint Review & Retrospective (Friday): Each member gave a short update on completed tasks, current progress, and any blockers. Technical progress was shared through screen sharing where needed. The session included a discussion on what worked well, what could be improved, and what actions to take next. Key decisions, feedback, and action items were recorded before each meeting ended.

Milestone Reviews: Held before each submission. All documents were reviewed against the rubric, and sign-off was required from all members before the Canvas upload.

Minutes were recorded in a shared Google Doc (Group3_Minutes_YYYY-MM-DD), distributed by the Communication Leader within 24 hours via WhatsApp and Teams, and archived in the GitHub Wiki.

3. Team Communication

Channel	Purpose	Response Time	Owner
WhatsApp Group	Day-to-day messages and urgent flags	Within 4 hours (working day)	All members
MS Teams Channel	Formal discussion, file sharing, meeting recordings	Within 8 hours	Eben (Comms Lead)

Channel	Purpose	Response Time	Owner
GitHub Issues / PRs	Code review, bug tracking, technical discussion	Within 24 hours	Arafat (GitHub Mgr)
GitHub Projects Board	Sprint tracking, task progress, blockers	Updated daily	Josh (Scrum Master)
UH Email	Module tutor correspondence, formal submissions	Within 24 hours	Tamanna (Team Lead)

All important project decisions were documented through Microsoft Teams discussions or GitHub issues, so there is a clear written record for the whole team. Decisions discussed verbally during meetings were only considered final once recorded and shared with the team.

- Any code changes had to be submitted through a Pull Request that included a clear description of the update, a linked GitHub Issue where applicable, and at least one approval from another team member before being merged into main. This helped maintain code quality and version control.
- If a team member was unable to complete a task or meet a deadline, they were expected to inform the Scrum Master at least 48 hours in advance so responsibilities could be adjusted.
- Disagreements were first discussed during the Friday retrospective meeting. If the issue remained unresolved, it was escalated to the module tutor within the following sprint.
- Respectful, inclusive communication was required at all times across all channels.

4. Methods of Working Together

4.1 Agile Sprint Process

Two-week sprints aligned to the module timeline. Work was broken down into GitHub issues labeled by area (data/model/app/docs/test), sized using T-shirt sizing (S/M/L/XL), and assigned a single owner.

4.2 GitHub Workflow

- Main branch is protected – all changes via feature branches (feature/<issue-no>-<description>).
- Commits use conventional prefixes: feat:, fix:, docs:, test:, refactor:.
- PRs must include: description, linked Issue, test evidence and confirmation of Flake8/Black pass.

4.3 Coding Standards

Python code follows PEP 8, auto formatted with Black and linted with Flake8. All functions include Google-style docstrings. Random seed fixed to 42 throughout (supports KPI 10: 100% reproducible pipeline). Notebooks converted to .py scripts via nbconvert before PR submission.

4.4 Shared File Structure

To maintain organisation and reproducibility, the team used a shared folder structure within the GitHub repository.

/data/raw/ – Stores the original datasets collected from external sources. These files were kept unchanged to preserve the integrity of the source data.

/data/processed/ – Contains cleaned and preprocessed datasets, feature-engineered data and final feature matrices used for model training and evaluation.

/models/ – Stores trained and versioned machine learning models, including CNN, ResNet50, Hybrid ResNet50 + XGBoost, and baseline Random Forest models.

/experiments/ – Contains experiment tracking files such as Optuna logs, hyperparameter tuning results, model performance summaries, and experiment configurations. A minimum of 20 tuning experiments were recorded for each model to support KPI tracking and reproducibility. We ran 25 Optuna trials for the hybrid model.

/docs/plans/ – Stores all planning documents, including the Project Specifications, Timeline, Data Management Plan, Pipeline Workflow, Test Plan, and Team Plan.

4.5 Absence and Contingency

If a member was absent, their sprint tasks were redistributed at the next stand-up. A bus-factor policy required at least two members to be familiar with every critical module, achieved through pair programming and mandatory code review.

4.6 Decision-Making

Day-to-day technical decisions (library choice and hyperparameter ranges) were made by the role owner and documented in a GitHub issue comment. Architectural decisions (model fusion strategy and app framework) required a 4/6 majority vote. Assessment scope changes required unanimous agreement and module tutor sign-off.

5. Project Completion Update

This section was added at the end of the project to summarize what was actually built and delivered.

Final Model: The hybrid ResNet50 + XGBoost model was the final chosen model. ResNet50 was used for transfer learning instead of EfficientNet-B0 because residual connections worked better for the abstract patterns in GAF images. The 2048-dim embeddings from ResNet50's global average pooling layer were concatenated with 80 tabular features and passed to XGBoost.

Hyperparameter Tuning: 25 Optuna Bayesian optimization trials were completed for the hybrid model.

Final Test Set Results: Weighted F1 = 0.9822, AUC-ROC = 0.9903, Accuracy = 98.18%, High-Risk Recall = 1.0000. All four KPIs passed.

App: Deployed as a Streamlit web application running on Google Colab via localtunnel. Users enter a ticker symbol and get a risk classification plus a SHAP bar chart showing the top 15 features that drove the prediction.

Dataset: 364 S&P 500 companies were used after merging all four datasets and removing rows with missing values.